# SysBook

Sistema de Gestão de Doações para Biblioteca Comunitária

Autor: FRANCISCO ANTONIO DE ARAÚJO E SOUZA

UNINTER - Ciência da Computação

RU: 5149011

Um sistema web desenvolvido em JavaScript/Node.js para gerenciar doações de livros em bibliotecas comunitárias.

- **YouTube:**

https://www.youtube.com/playlist?list=PL05lqD2VYpzzbEgs17b7ptrjn8w0MjdA2

- **GitHub:**

`https://github.com/proffaraujo/sysbook.git`

SysBook

# Sistema de Gestão de Doações para Biblioteca

**Versão:** 1.0.0
**Licença:** MIT

---

## VISÃO GERAL

O Sistema de Gestão de Doações é uma solução enterprise completa para bibliotecas que desejam gerenciar doações de livros de forma eficiente, transparente e impactante. O sistema oferece desde o registro inicial das doações até relatórios executivos de impacto social.

### PRINCIPAIS FUNCIONALIDADES

- Gestão completa de doadores e histórico de doações
- Catalogação inteligente de livros com validação ISBN
- Sistema de campanhas com metas e acompanhamento
- Geração automática de recibos para doadores
- Dashboard executivo com métricas de impacto
- Relatórios avançados em PDF e Excel
- Sistema de notificações multi-canal
- Segurança robusta com JWT e auditoria completa

---

# ARQUITETURA MODULAR

O sistema foi arquitetado em 7 módulos independentes para máxima flexibilidade, manutenibilidade e escalabilidade:

**FLUXO DE INTEGRAÇÃO:** 1A (Core Essencial) → 1B (Administração) → 2A (Campanhas e Recibos) → 2B (Estatísticas e Relatórios) → 3A (Segurança e Dados) → 3B (Comunicação e Logs)

## MÓDULOS DO SISTEMA

### MÓDULO 1A - CORE ESSENCIAL

- Responsabilidade: Entidades fundamentais
- Principais Classes: Doador, Doacao, Livro, ItemDoacao
- Uso Recomendado: Apresentações executivas

### MÓDULO 1B - ADMINISTRAÇÃO

- Responsabilidade: Usuários e DTOs
- Principais Classes: Administrador, Result, DadosDoacao
- Uso Recomendado: Reuniões de desenvolvimento

### MÓDULO 2A - CAMPANHAS E RECIBOS

- Responsabilidade: Funcionalidades principais
- Principais Classes: Campanha, Recibo, MetaTipo
- Uso Recomendado: Reuniões de negócio

### MÓDULO 2B - ESTATÍSTICAS E RELATÓRIOS

- Responsabilidade: Analytics e dashboard
- Principais Classes: PainelImpacto, EstatisticasGerais, RelatorioImpacto
- Uso Recomendado: Análise funcional

### MÓDULO 3A - SEGURANÇA E DADOS

- Responsabilidade: Auth e persistência
- Principais Classes: AuthenticationService, DatabaseConnection
- Uso Recomendado: Implementação de segurança

### MÓDULO 3B - COMUNICAÇÃO E LOGS

- Responsabilidade: Integrações e logs
- Principais Classes: EmailService, NotificationService, LoggingService
- Uso Recomendado: Implementação de integrações

---

# INÍCIO RÁPIDO

## PRÉ-REQUISITOS

• Node.js 18+ ou Java 11+
• PostgreSQL 13+
• Redis 6+ (opcional, para cache)
• SMTP configurado (para emails)

## INSTALAÇÃO RÁPIDA

### 1. Clone o repositório

```
git clone https://github.com/biblioteca/sistema-gestao-doacoes.git
cd sistema-gestao-doacoes
```

### 2. Instale as dependências

```
npm install
```
ou
```
mvn clean install
```

### 3. Configure o banco de dados

```
cp .env.example .env
```

Edite o arquivo .env com suas configurações

### 4. Execute as migrações

```
npm run migrate
ou
mvn flyway:migrate
```

### 5. Inicie o servidor

```
npm start
ou
mvn spring-boot:run
```

## ACESSO AO SISTEMA

• Interface Web: http://localhost:3000
• API REST: http://localhost:3000/api
• Documentação: http://localhost:3000/docs
• Monitoramento: http://localhost:3000/health

---

# DOCUMENTAÇÃO COMPLETA

## DIAGRAMAS UML POR CONTEXTO

### PARA DIFERENTES PÚBLICOS:

| PÚBLICO | MÓDULOS RECOMENDADOS | TEMPO | FOCO |
|---|---|---|---|
| Executivos | 1A | 5 min | Visão geral do negócio |
| Product Owners | 1A + 2A | 10 min | Funcionalidades principais |
| Desenvolvedores | 1A + 1B + 2A | 20 min | Entidades e operações |
| Analistas | 1A + 1B + 2A + 2B | 30 min | Funcionalidades completas |
| DevOps/Security | 1A + 1B + 3A | 40 min | Segurança e dados |
| Arquitetos | Todos os 7 módulos | 60 min | Arquitetura completa |

### LOCALIZAÇÃO DOS DIAGRAMAS:

```
docs/
├── uml/
│   ├── modulo-1a-core-essencial.puml
│   ├── modulo-1b-administracao.puml
│   ├── modulo-2a-campanhas-recibos.puml
│   ├── modulo-2b-estatisticas-relatorios.puml
│   ├── modulo-3a-seguranca-dados.puml
│   └── modulo-3b-comunicacao-logs.puml
├── images/
│   ├── modulo-1a-core-essencial.png
│   ├── modulo-1b-administracao.png
│   ├── modulo-2a-campanhas-recibos.png
│   ├── modulo-2b-estatisticas-relatorios.png
│   ├── modulo-3a-seguranca-dados.png
│   └── modulo-3b-comunicacao-logs.png
└── README.md
```

---

# DETALHAMENTO TÉCNICO

## MÓDULO 1A: CORE ESSENCIAL

**RESPONSABILIDADES:** • Gestão de doadores e seu histórico
• Processo principal de doações
• Catalogação de livros com validação
• Relacionamento entre doações e itens

**PRINCIPAIS ENTIDADES:**

**Classe Doador:**

- id: number
- nomeCompleto: string
- email: string
- telefone: string
- ativo: boolean

**Classe Doacao:**

- id: number
- doadorId: number
- status: StatusDoacao
- protocolo: string
- valorEstimado: decimal

**APIS PRINCIPAIS:** • POST /api/doadores - Cadastro de doadores
• POST /api/doacoes - Registro de doações
• GET /api/livros - Consulta de livros
• POST /api/itens-doacao - Catalogação de itens

## MÓDULO 1B: ADMINISTRAÇÃO

**RESPONSABILIDADES:** • Gestão de usuários administrativos
• DTOs para transferência de dados
• Padrões de retorno padronizados
• Filtros e configurações

**PRINCIPAIS CLASSES:**

**Classe Administrador:**

- id: number
- username: string
- nome: string
- email: string
- ativo: boolean

**Classe Result:**

- success: boolean
- message: string
- data: T
- errorCode?: string

**APIS PRINCIPAIS:** • POST /api/auth/login - Autenticação
• POST /api/admin/usuarios - Gestão de usuários
• GET /api/admin/configuracoes - Configurações do sistema

## MÓDULO 2A: CAMPANHAS E RECIBOS

**RESPONSABILIDADES:** • Sistema de campanhas com metas
• Geração automática de recibos
• Acompanhamento de progresso
• Configurações de campanha

**PRINCIPAIS CLASSES:**

**Classe Campanha:**

- id: number
- titulo: string
- metaLivros: number
- metaDoacoes: number
- metaValor: decimal
- ativa: boolean

**Classe Recibo:**

- id: number
- doacaoId: number
- numeroRecibo: string
- valorTotal: decimal

**APIS PRINCIPAIS:** • POST /api/campanhas - Criação de campanhas
• GET /api/campanhas/ativas - Campanhas ativas
• POST /api/recibos/gerar - Geração de recibos
• GET /api/recibos/{numero} - Consulta de recibos

## MÓDULO 2B: ESTATÍSTICAS E RELATÓRIOS

**RESPONSABILIDADES:** • Dashboard executivo centralizado
• Estatísticas consolidadas do sistema
• Relatórios em múltiplos formatos
• Gráficos e visualizações

**PRINCIPAIS CLASSES:**

**Classe PainelImpacto:**

- getInstance(): PainelImpacto (Singleton)
- atualizarEstatisticas(): boolean
- gerarRelatorioImpacto(): RelatorioImpacto

**Classe EstatisticasGerais:**

- totalDoacoes: number
- totalDoadores: number
- totalLivros: number
- valorTotalEstimado: decimal

**APIS PRINCIPAIS:** • GET /api/dashboard/estatisticas - Estatísticas gerais
• GET /api/relatorios/impacto - Relatório de impacto
• POST /api/relatorios/gerar - Geração de relatórios
• GET /api/graficos/dados - Dados para gráficos

## MÓDULO 3A: SEGURANÇA E DADOS

**RESPONSABILIDADES:** • Autenticação JWT com blacklist
• Pool de conexões com transações
• Cache Redis para performance
• Configurações de segurança

**PRINCIPAIS CLASSES:**

**Classe AuthenticationService:**

- gerarToken(usuario: Administrador): string
- validarToken(token: string): Result
- invalidarToken(token: string): boolean

**Classe DatabaseConnection:**

- executarQuery(sql: string, params: any[]): Result
- iniciarTransacao(id: string): Transaction
- commitTransacao(id: string): boolean

**APIS PRINCIPAIS:** • POST /api/auth/token - Geração de tokens
• POST /api/auth/logout - Invalidação de tokens
• GET /api/auth/validate - Validação de tokens
• GET /api/database/health - Status do banco

## MÓDULO 3B: COMUNICAÇÃO E LOGS

**RESPONSABILIDADES:** • Envio de emails e notificações
• Sistema de logs com rotação
• Gestão de arquivos e uploads
• Múltiplos canais de comunicação

**PRINCIPAIS CLASSES:**

**Classe EmailService:**

- enviarRecibo(dadosRecibo: DadosRecibo): Result
- enviarNotificacao(destinatario: string, assunto: string): Result

**Classe LoggingService:**

- log(nivel: LogLevel, mensagem: string): void
- obterLogs(filtros: FiltrosLog): LogEntry[]

**APIS PRINCIPAIS:** • POST /api/email/enviar - Envio de emails
• POST /api/notificacoes/enviar - Notificações
• POST /api/arquivos/upload - Upload de arquivos
• GET /api/logs - Consulta de logs

---

# SEGURANÇA

## AUTENTICAÇÃO E AUTORIZAÇÃO

• JWT Tokens com refresh automático
• Blacklist de tokens para logout seguro
• Controle de acesso baseado em roles
• Auditoria completa de operações

## PROTEÇÃO DE DADOS

• Criptografia de senhas com bcrypt
• Validação rigorosa de inputs
• Rate limiting para APIs
• HTTPS obrigatório em produção

---

# MONITORAMENTO E OBSERVABILIDADE

## MÉTRICAS PRINCIPAIS

• Performance: Tempo de resposta das APIs
• Negócio: Doações por período, valor arrecadado
• Sistema: CPU, memória, conexões de banco
• Usuário: Sessões ativas, operações por usuário

## LOGS E RASTREAMENTO

### Exemplo de log estruturado:

```
{
  "timestamp": "2025-08-15T10:30:00Z",
  "level": "INFO",
  "service": "doacao-service",
  "operation": "registrar-doacao",
  "userId": "admin123",
  "doadorId": "doador456",
  "message": "Doação registrada com sucesso",
  "duration": "150ms"
}
```

---

# DEPLOYMENT

## DOCKER

### Arquivo docker-compose.yml:

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/doacoes
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis

  db:
    image: postgres:13
    environment:
      POSTGRES_DB: doacoes
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass

  redis:
    image: redis:6-alpine
```

## CLOUD NATIVE

• Kubernetes ready com health checks
• Auto-scaling baseado em métricas
• Circuit breakers para resilência
• Blue-green deployment para zero downtime

---

# TESTES

## COBERTURA DE TESTES

• Unitários: 95%+ cobertura
• Integração: APIs e banco de dados
• E2E: Fluxos principais de usuário
• Performance: Load testing com K6

**Comandos para executar testes:**

```
npm test                    # Testes unitários
npm run test:integration    # Testes de integração
npm run test:e2e            # Testes end-to-end
npm run test:performance    # Testes de performance
```

---

# CONTRIBUINDO

## GUIA DE CONTRIBUIÇÃO

1. Fork o repositório
2. Crie uma branch para sua feature (git checkout -b feature/nova-funcionalidade)
3. Commit suas mudanças (git commit -am 'Adiciona nova funcionalidade')
4. Push para a branch (git push origin feature/nova-funcionalidade)
5. Abra um Pull Request

## PADRÕES DE CÓDIGO

• ESLint/Prettier para formatação
• Conventional Commits para mensagens
• TDD para novas funcionalidades
• Code Review obrigatório

---

# SUPORTE

## CANAIS DE SUPORTE

• YouTube: https://www.youtube.com/playlist?list=PL05lqD2VYpzzbEgs17b7ptrjn8w0MjdA2
• GitHub: https://github.com/proffaraujo/sysbook.git

• Issues: GitHub Issues
• Discussions: GitHub Discussions
• Email: sysbook@gmail.com

## REPORTAR BUGS

Use o template de issue do GitHub incluindo:

• Versão do sistema
• Passos para reproduzir
• Comportamento esperado vs atual
• Screenshots (se aplicável)

---

# LICENÇA

Este projeto está licenciado sob a MIT License - veja o arquivo LICENSE para detalhes.

---

# AGRADECIMENTOS

• Biblioteca parceira da comunidade do Jangurussu pelos *feedbacks* e apoio

• À Profa. LIZIANE HOBMEIR e à UNINTER, pela orienta e oportunidade que tornam este projeto possível.

• Comunidade Open Source pelas contribuições

---

Este projeto tem como objetivo transformar bibliotecas comunitárias através de tecnologias da Ciência da Computação.

Se este projeto lhe ajudar e contribuir com a comunidade em que está disponível, por favor, considere dar uma estrela no GitHub e no YouTube!

# SysBook – APÊNDICES

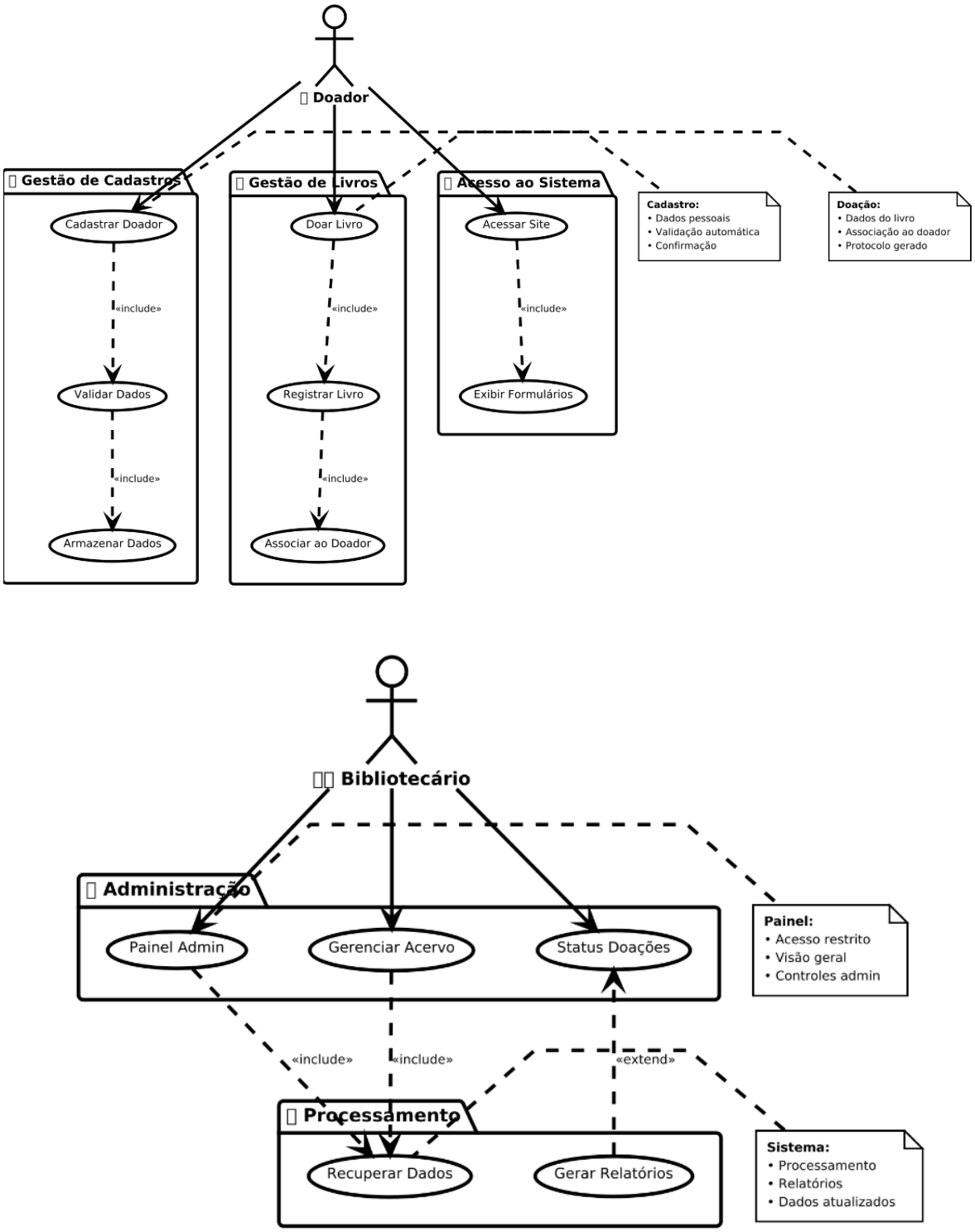



**Figura 01 - Diagrama de Caso de Uso (UML)**

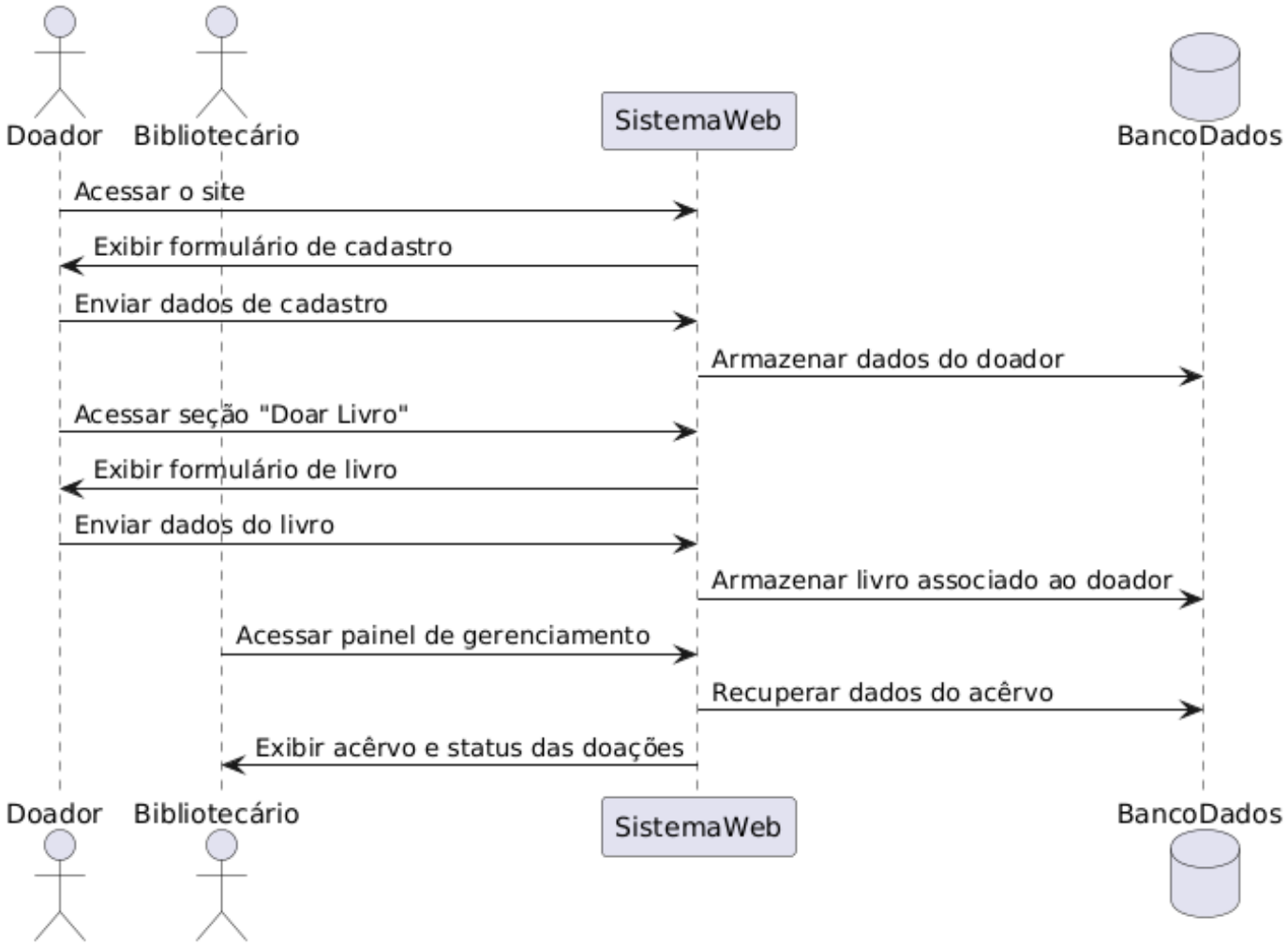



**Figura 02 - Diagrama de Seqência (UML)**

## Diagramas de Classes (UML)

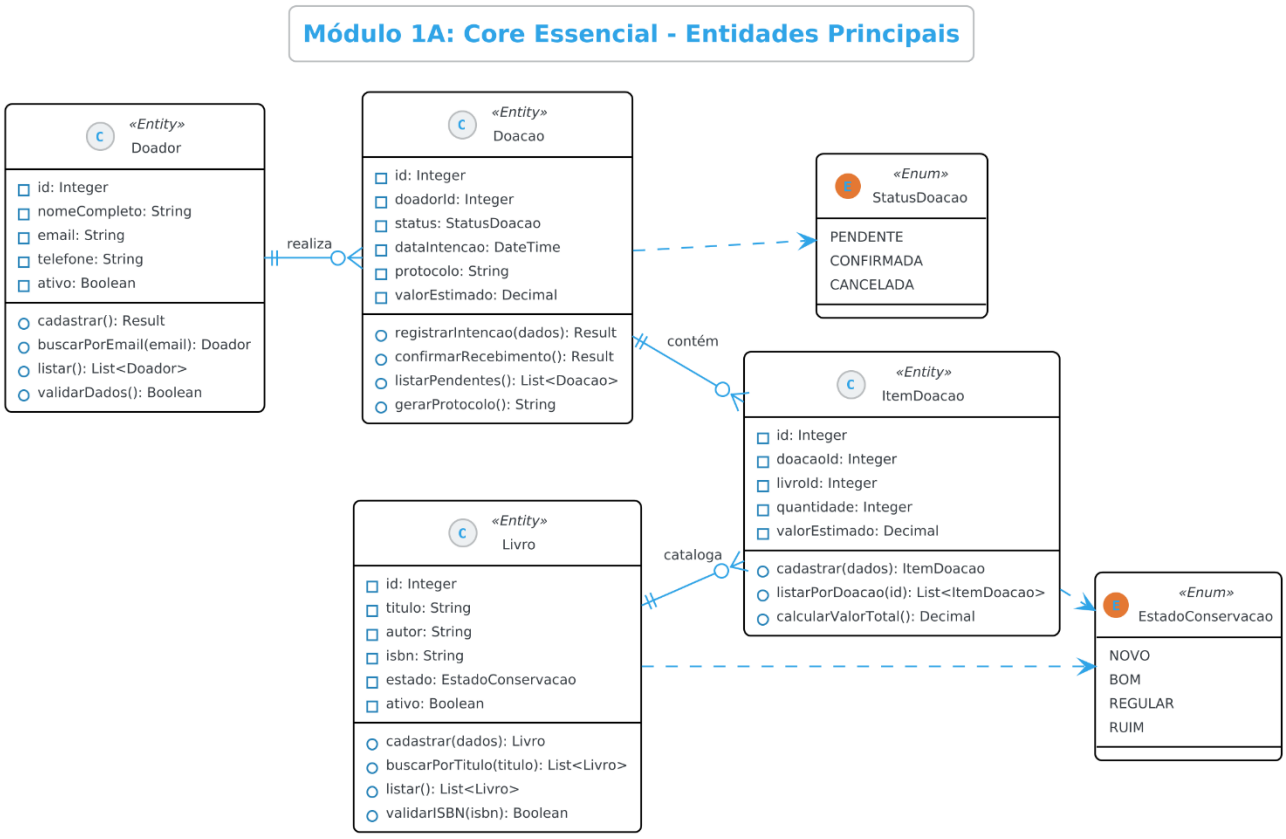



**Figura 03-1A - Diagramas de Classes (UML)**

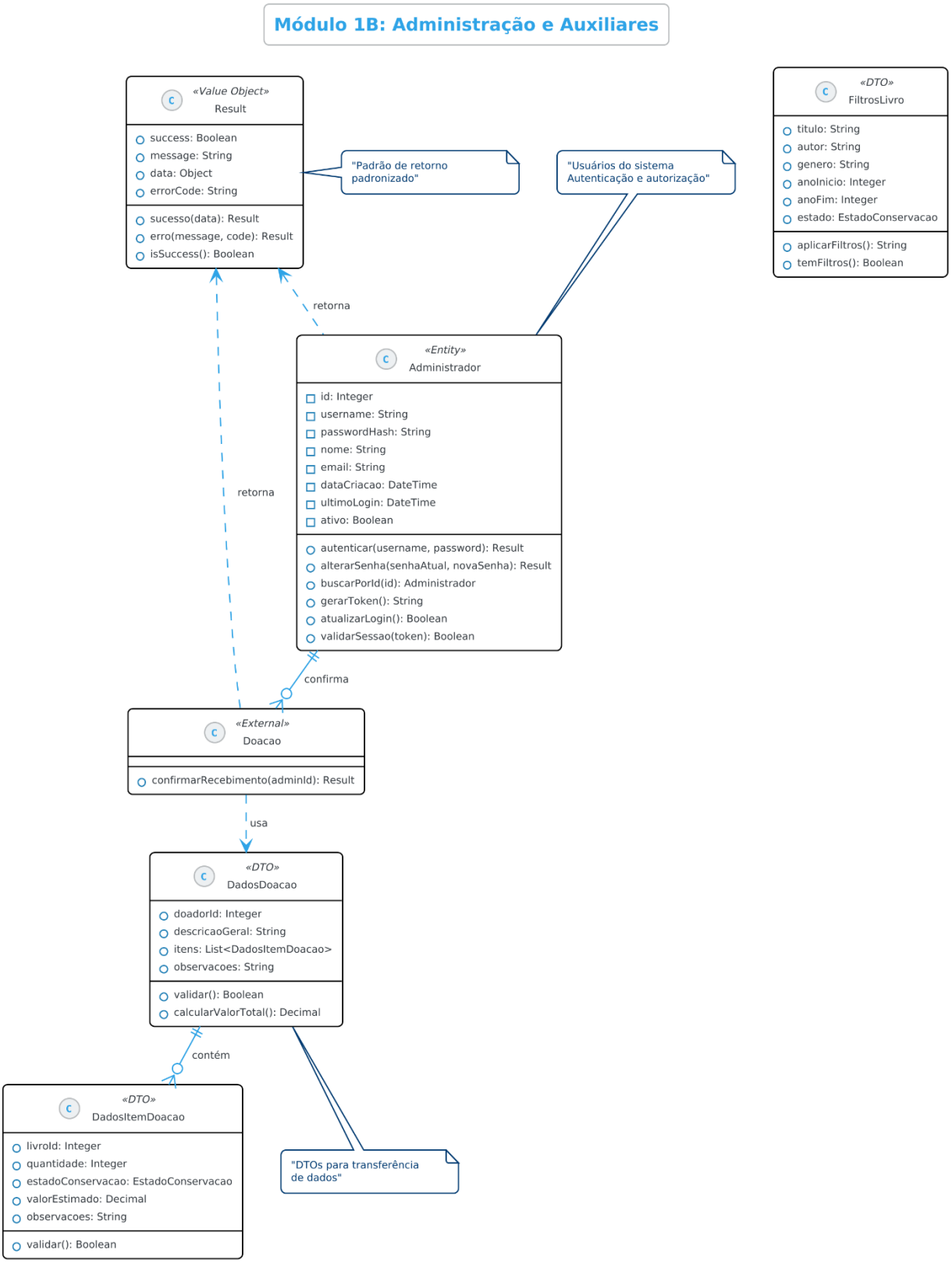



**Figura 03-1B - Diagramas de Classes (UML)**

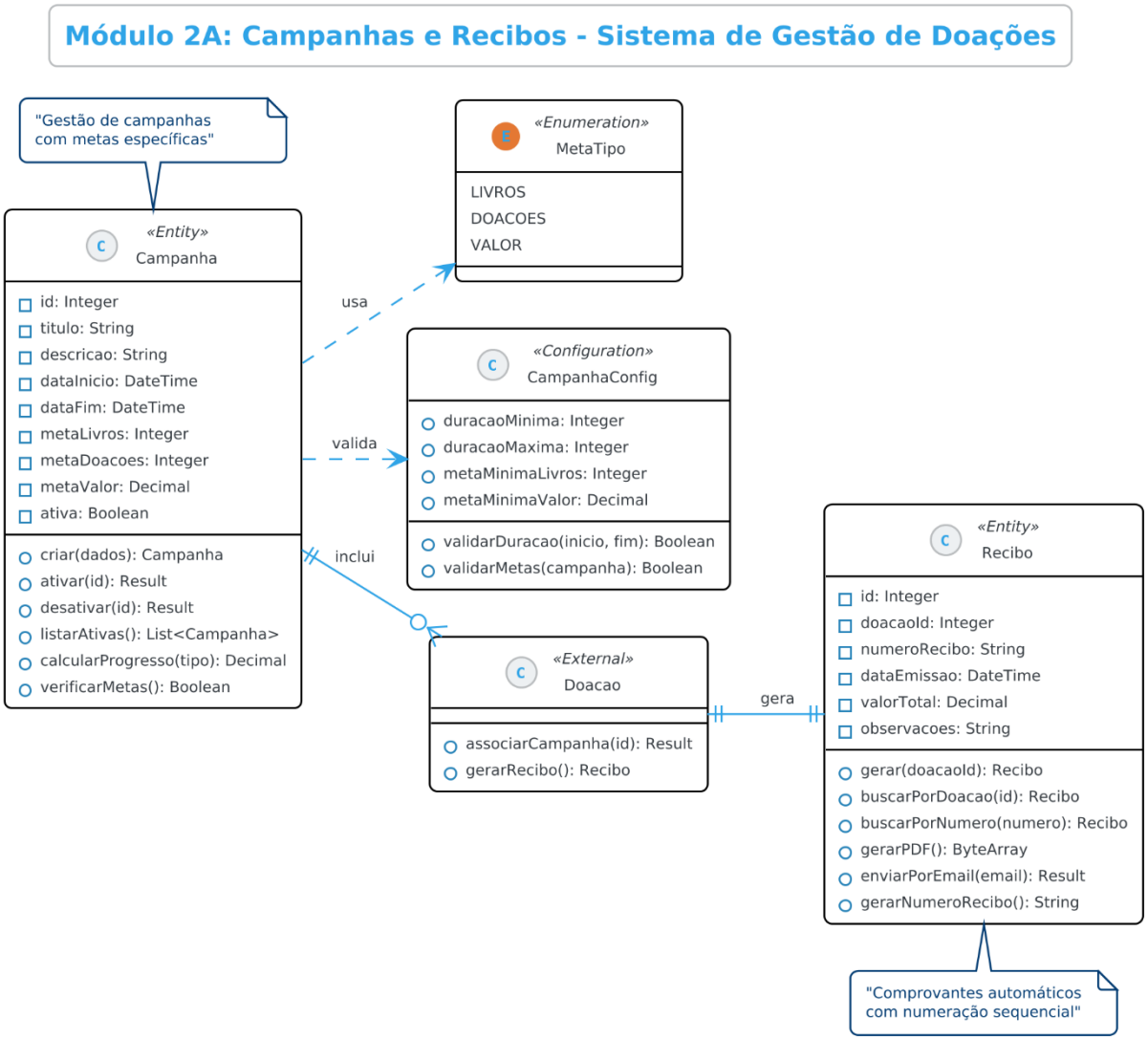



**Figura 03-2A - Diagramas de Classes (UML)**

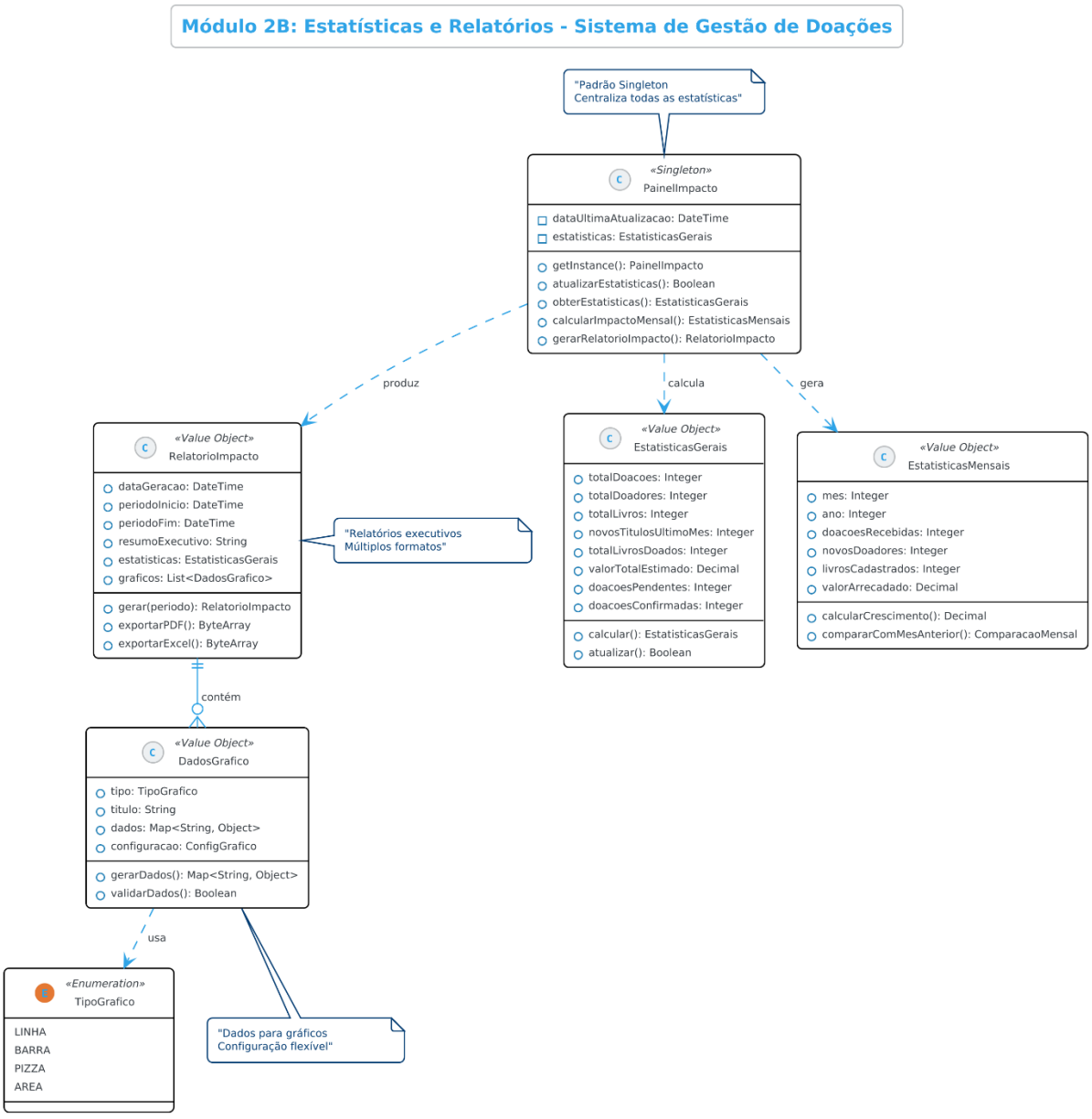



**Figura 03-2B - Diagramas de Classes (UML)**

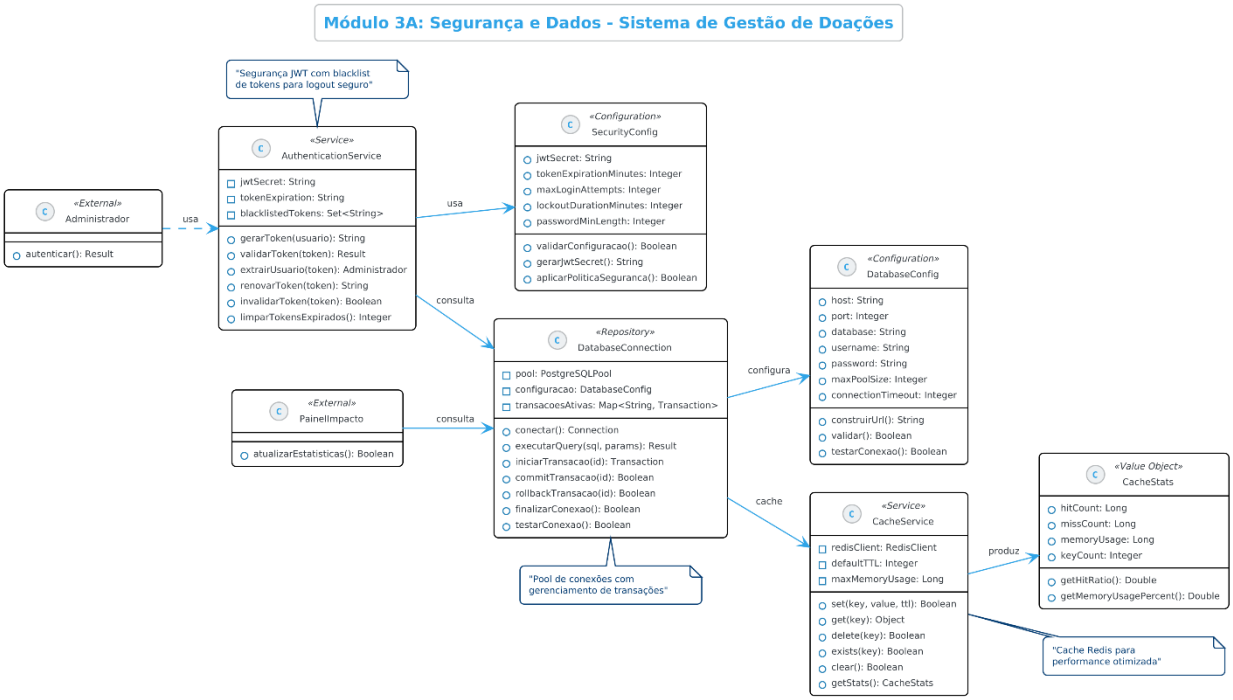



**Figura 03-3A - Diagramas de Classes (UML)**

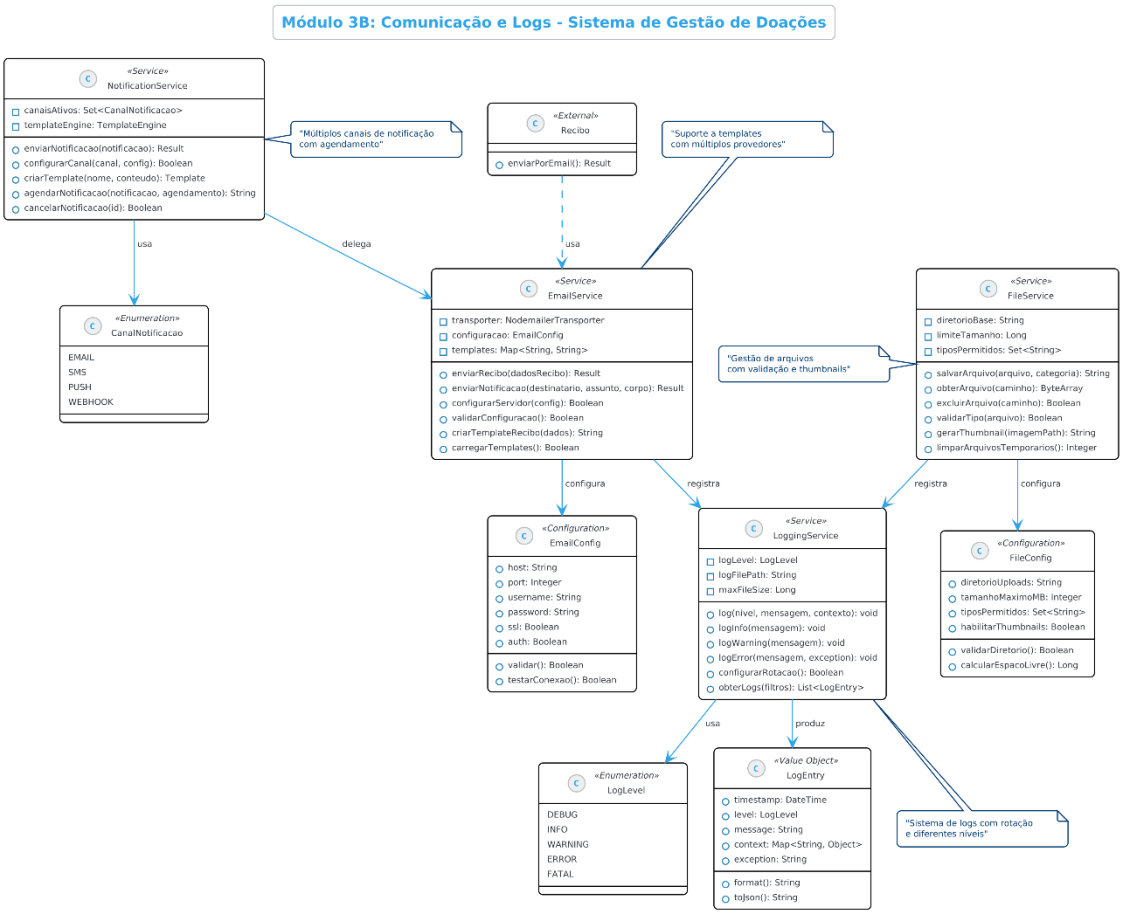



**Figura 03-3B - Diagramas de Classes (UML)**

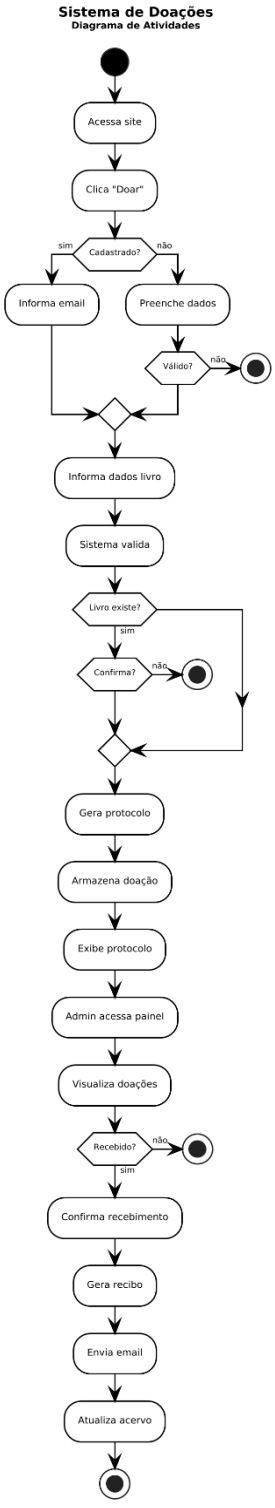



**Figura 04 - Diagrama de Atividades (UML)**

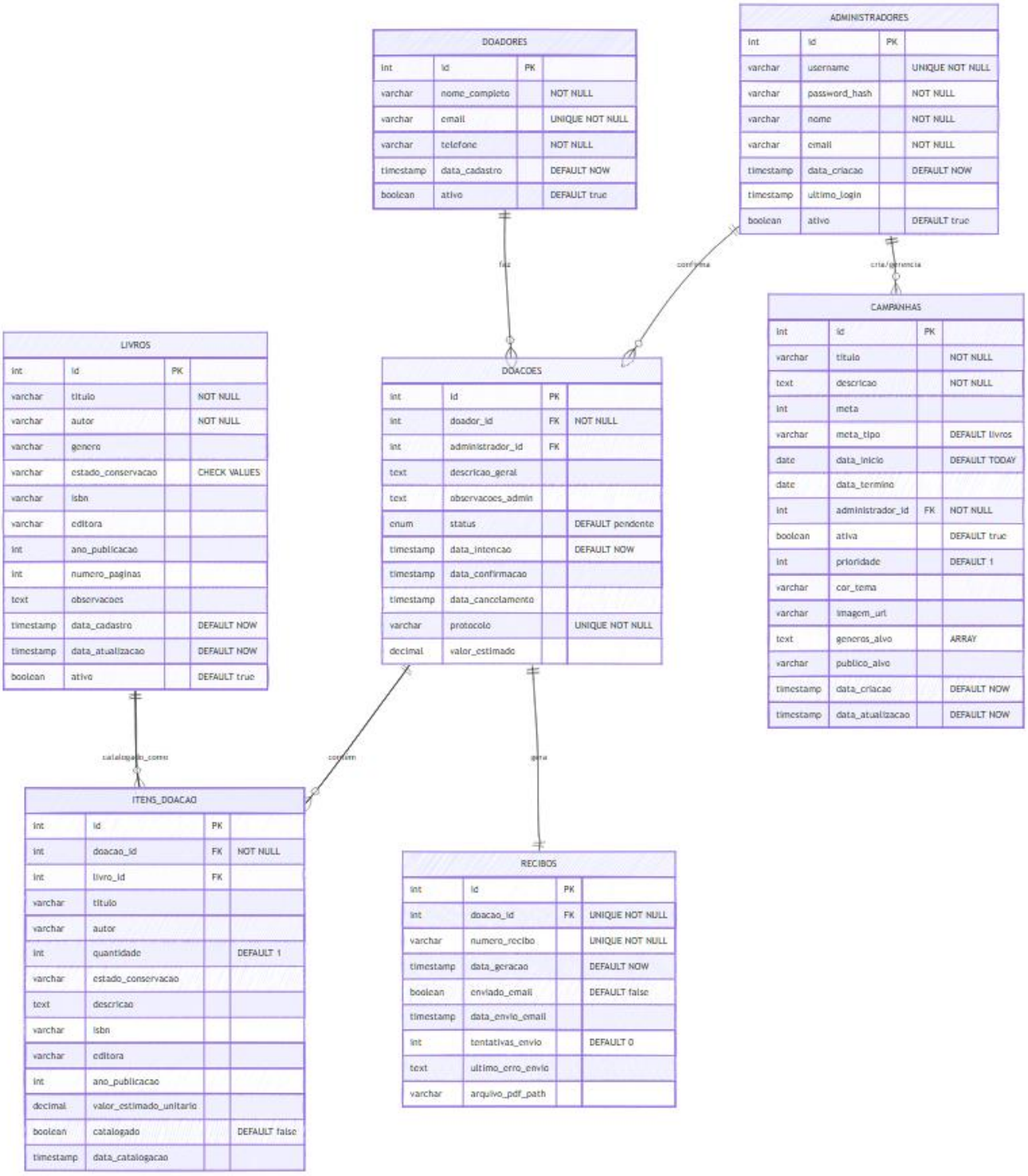



**Figura 05 - Diagrama de Entidades e Relacionamentos (DER)**

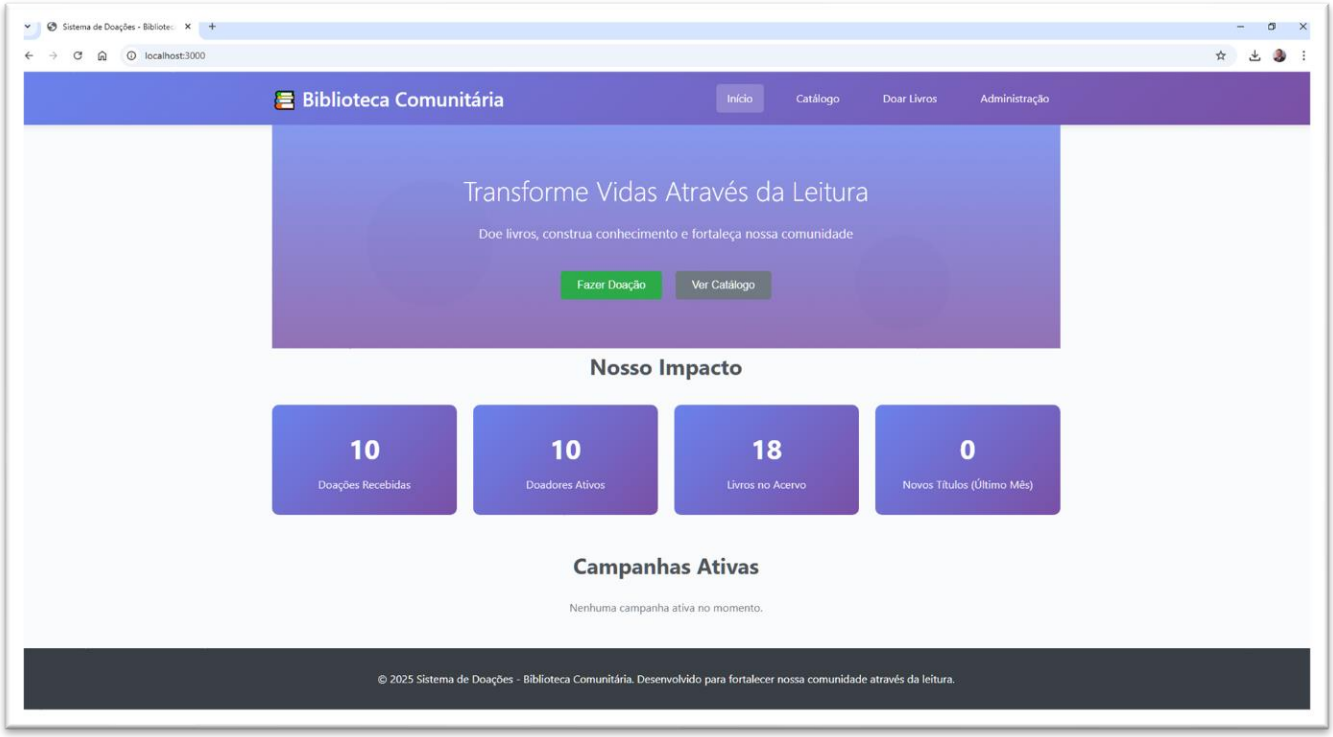



**Figura 06 – Home Page**

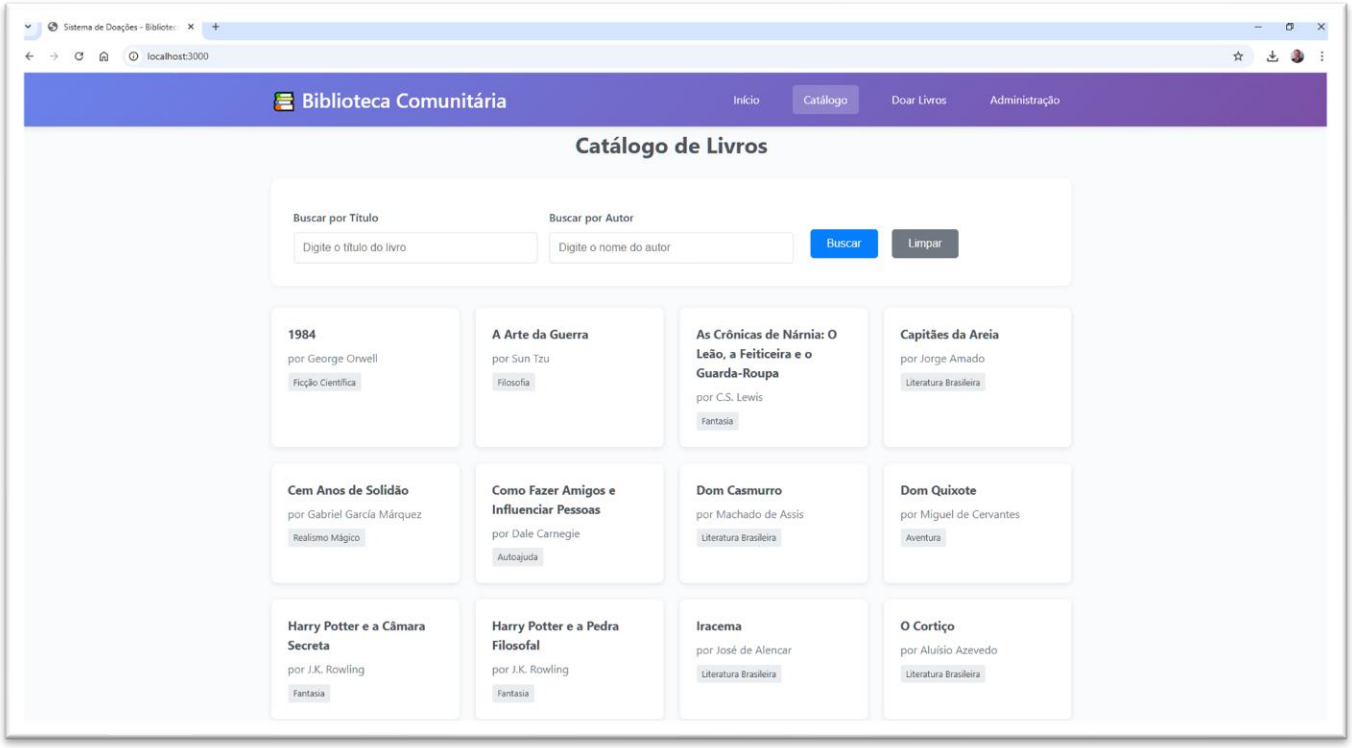



**Figura 07 – Tela Catálogo de livros**

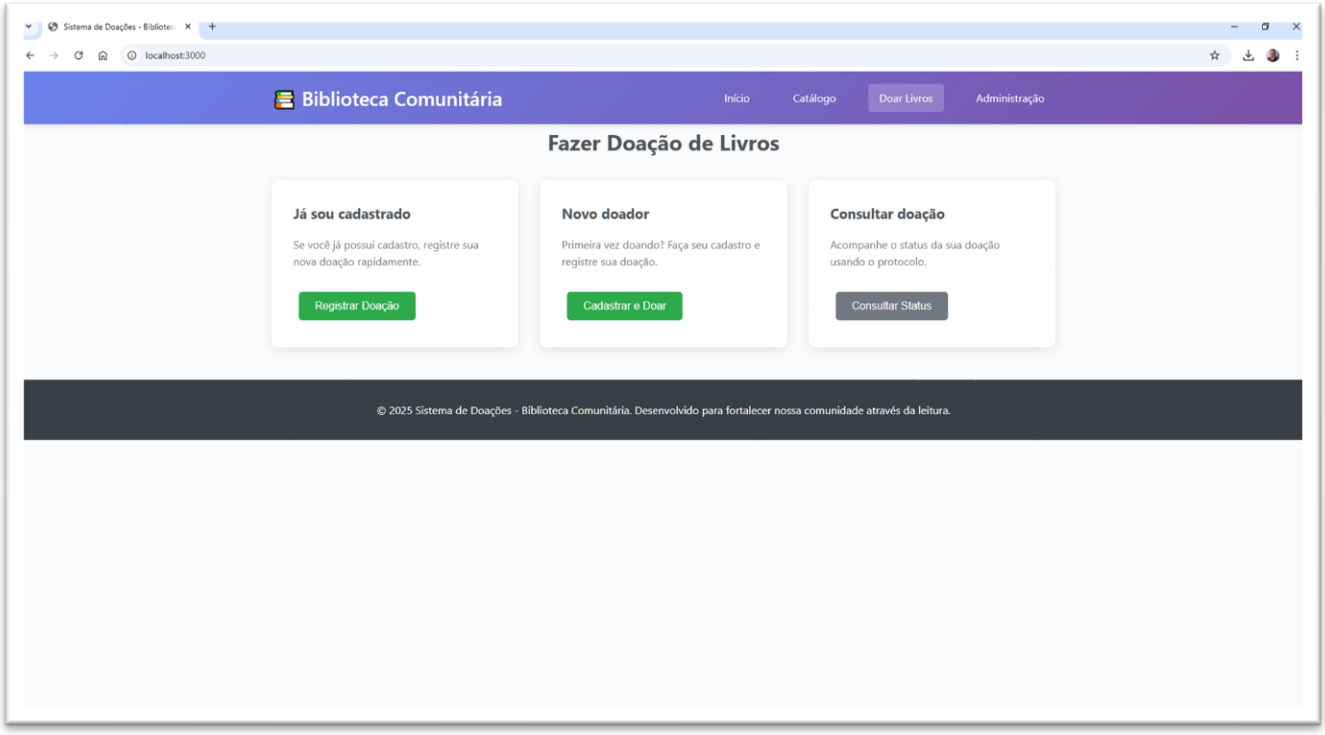



**Figura 08 – Tela de Doação de Livros**

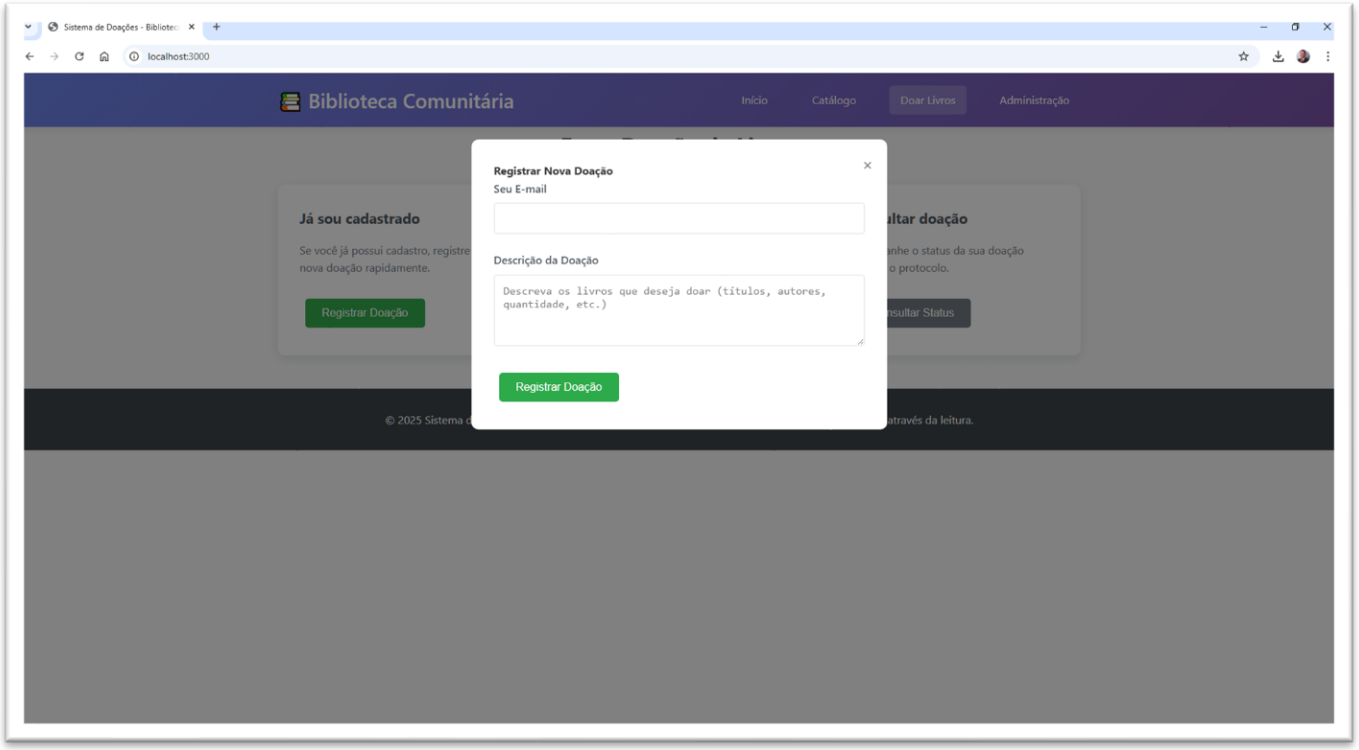



**Figura 09 – Tela de Doação de Livros – Registrar Nova Doação**

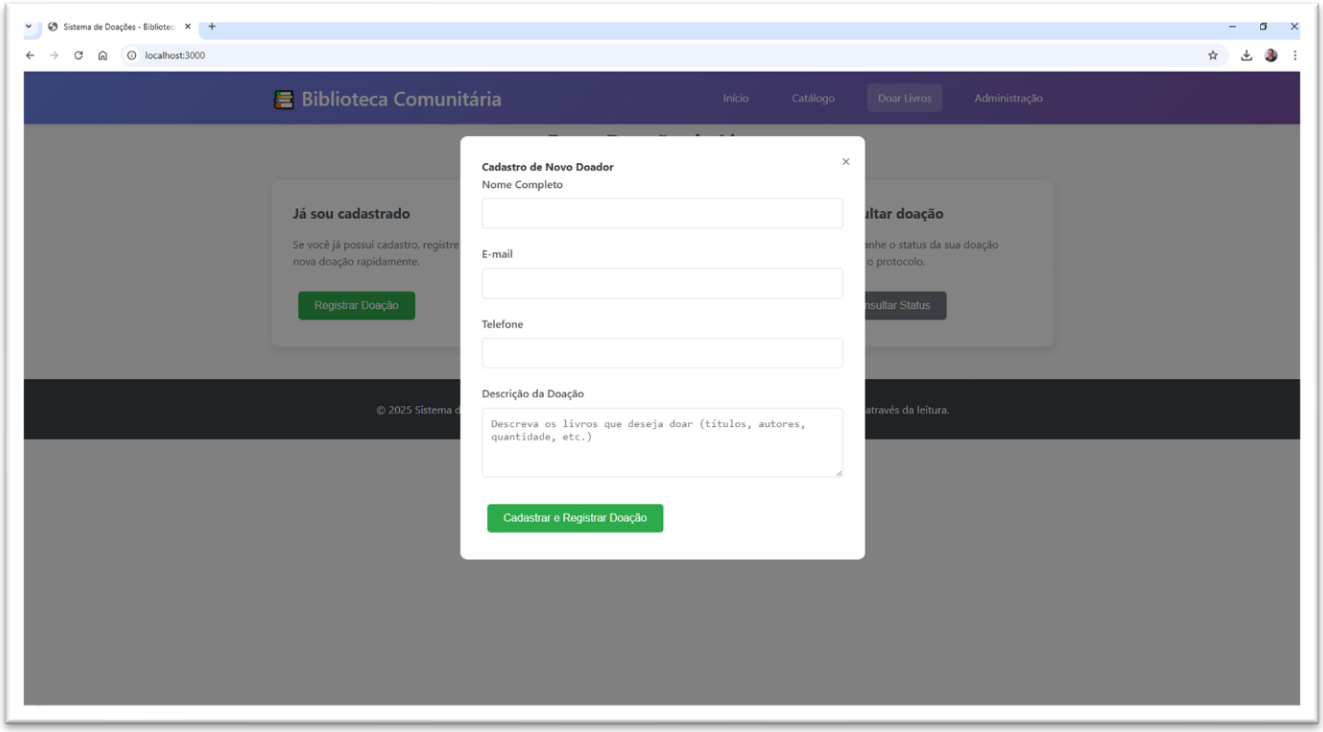



**Figura 10 – Tela de Doação de Livros – Cadastrar Novo Doador**

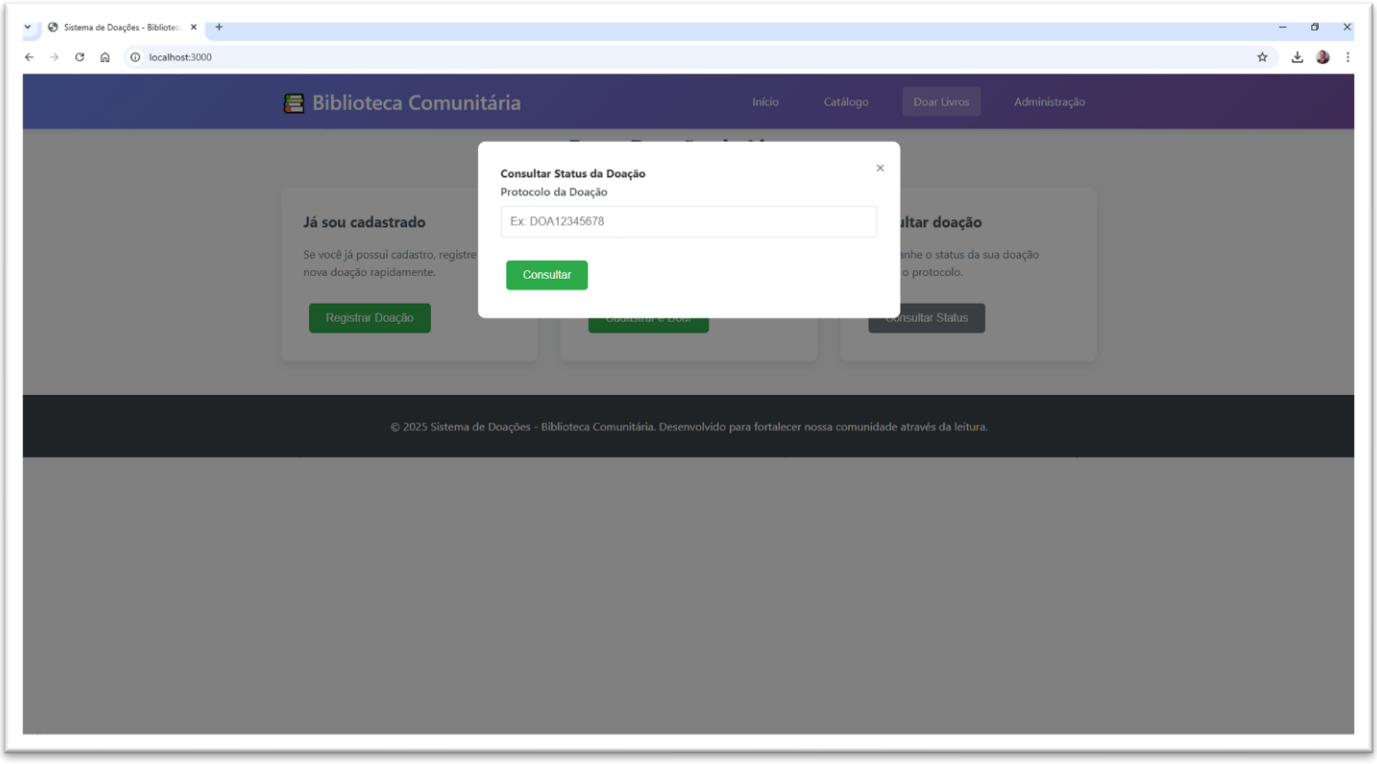



**Figura 11 – Tela de Doação de Livros – Consultar Status da Doação**

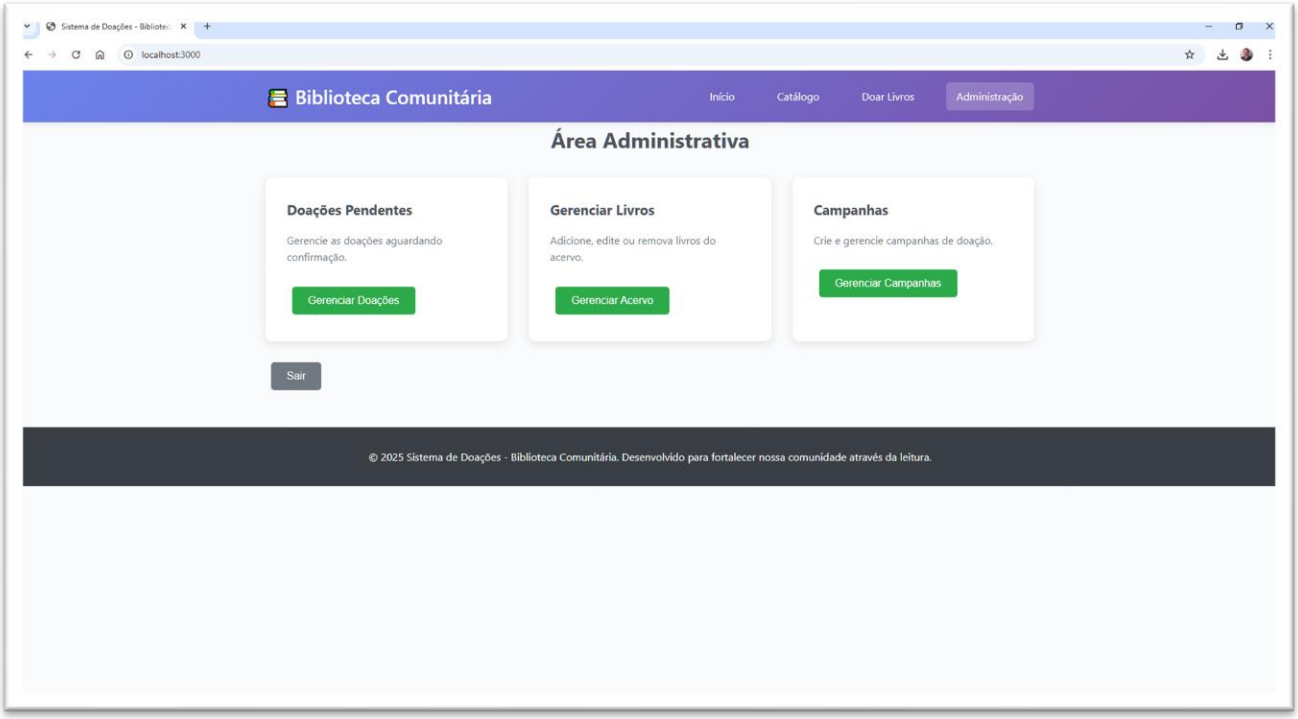



**Figura 12 – Tela de Operações Administrativas do Sistema**

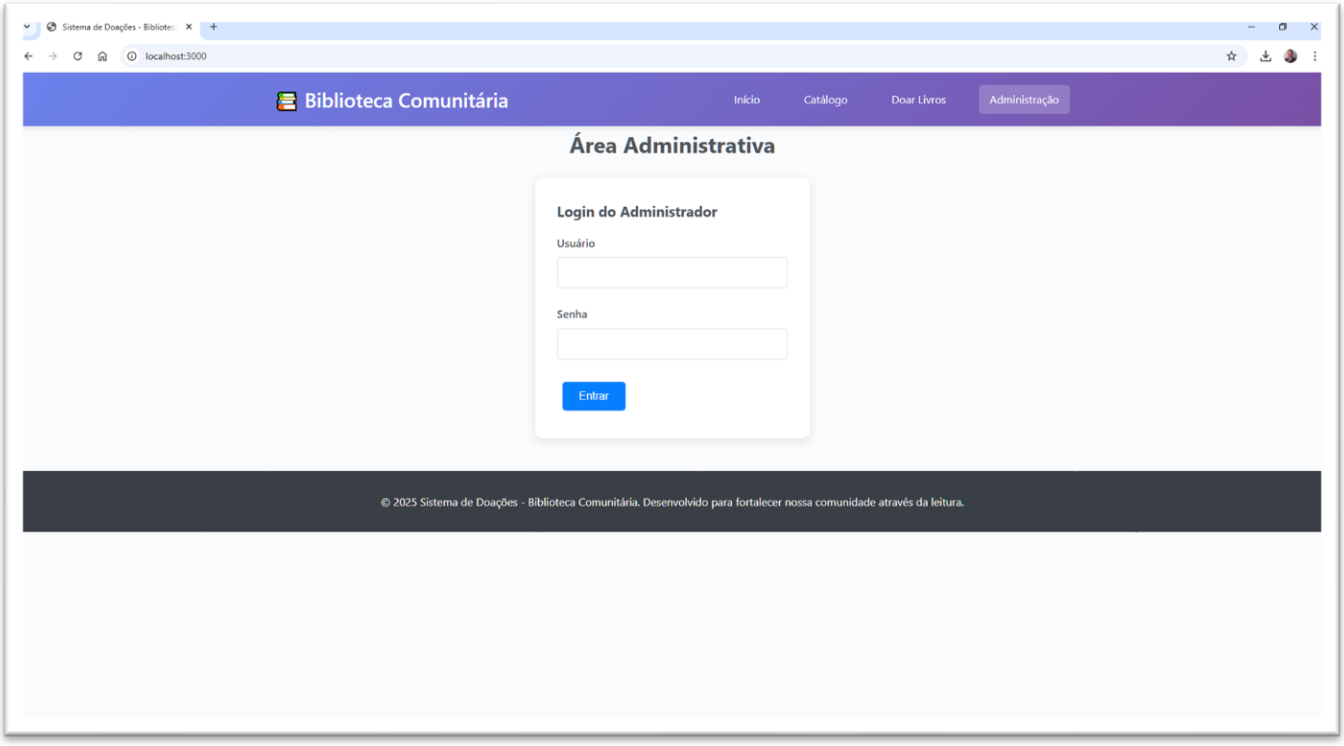



**Figura 13 – Tela de Login do Administrador**

# BIBLIOGRAFIA


ARMS, William Y. Digital Libraries. Cambridge: MIT Press, 2000.

BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software Architecture in Practice. 4. ed. Boston: Addison-Wesley, 2021.

DUCKETT, Jon. HTML & CSS: Design and Build Websites. Indianapolis: Wiley, 2011.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2019.

FOWLER, Martin. UML Essencial: Um Breve Guia para a Linguagem-Padrão de Modelagem de Objetos. 3. ed. Porto Alegre: Bookman, 2005.

KERZNER, Harold. Gestão de Projetos: As Melhores Práticas. 3. ed. Porto Alegre: Bookman, 2017.

LAUDON, Kenneth C.; LAUDON, Jane P. Sistemas de Informação Gerenciais. 11. ed. São Paulo: Pearson, 2014.

PREECE, Jennifer; ROGERS, Yvonne; SHARP, Helen. Design de Interação: Além da Interação Humano-Computador. 3. ed. Porto Alegre: Bookman, 2013.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: Uma Abordagem Profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson, 2019.